

O.P.O : A la fin de cette leçon l'apprenant devra être capable de :

- ✓ Définir et utiliser la fonction de lecture
- ✓ Définir et d'utiliser les fonctions d'écritures
- ✓ Définir et utiliser les structures qu'offrent JavaScript

I. Fonctions du langage

L'affichage et la lecture des données se font via les fonctions JavaScript **alert()** et **prompt()**.

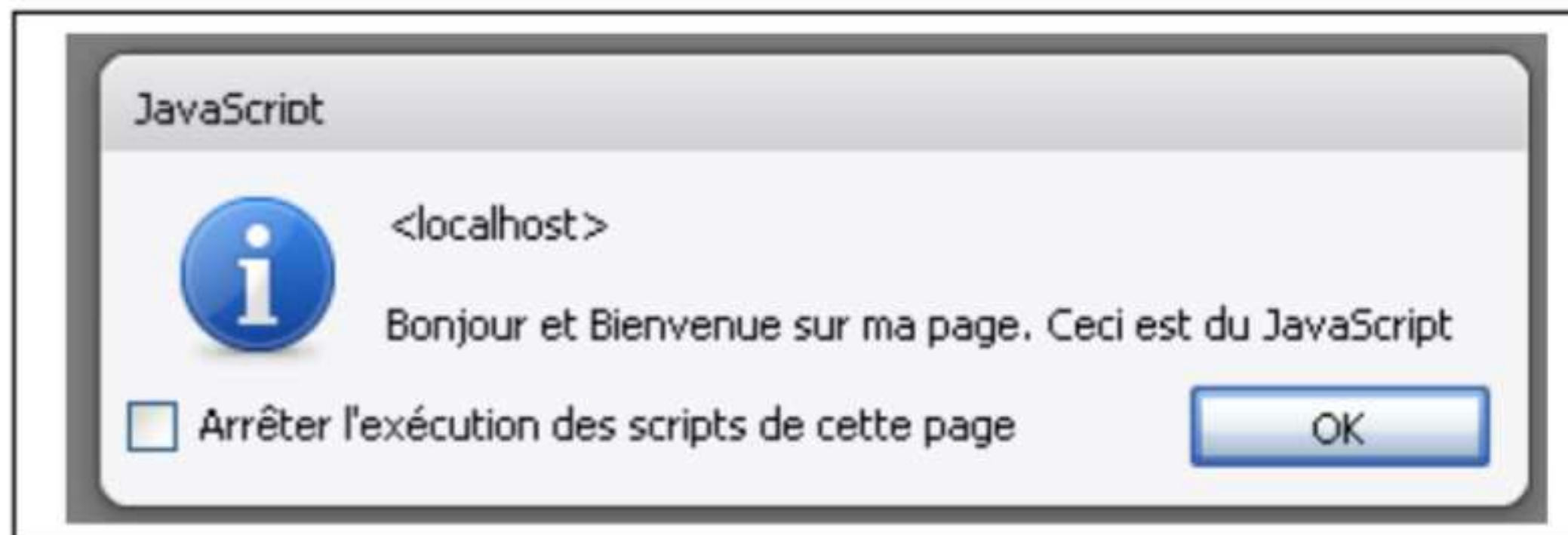
La fonction **alert ()** sert à afficher une valeur à l'écran et la fonction **prompt()** sert à lire une donnée au clavier.

Exemple :

➤ Syntaxes d’Affichage des données

- `alert (Nom_Variable)` : pour afficher le contenu d'une variable.
- `alert ('Mon Texte')` ou `alert ("Mon Texte")` : pour afficher un texte à l'écran.

Exemple d’affichage : `alert ("Bonjour et Bienvenue sur ma page. Ceci est du JavaScript");`

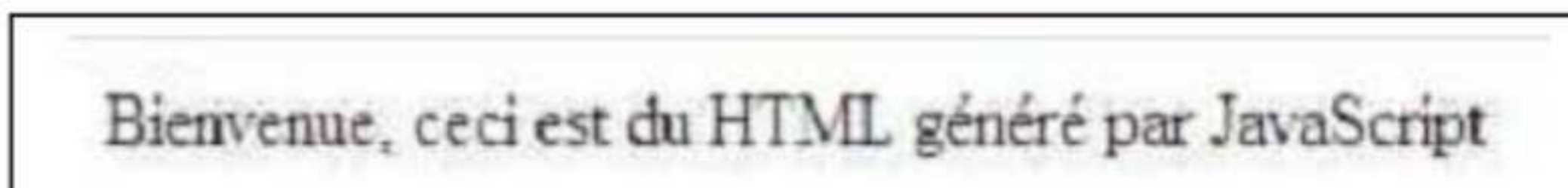


Remarque :

L'affichage peut aussi se faire grâce à la fonction **document.write('Votre Texte')** ou **document.write(Nom_Variable)**. Cette fonction génère du code HTML puis l'affiche.

Exemple : `document.write("Bienvenue, ceci est du HTML généré par JavaScript");`

Résultat :



➤ Lecture d’une donnée

Syntaxe : `X = prompt ('Texte à afficher')` ou `X = prompt ("Mon Texte")` : lire une donnée et la ranger dans la mémoire après validation de la saisie.

Exemple : `X = prompt ("Entrez votre nom");`

Résultat :



II. Structure en JavaScript

1- Structure Séquentielle

Pour exécuter des instructions en séquence, il suffit de les écrire chaque instruction les unes après les autres. Au cas où il y a plusieurs instructions sur une même ligne, il faut les faire suivre obligatoirement d'un point virgule.

Syntaxe :

```
Instruction 1
Instruction 2
Instruction3; Instruction4; Instruction5
Instruction 4 ;
```

Exemple :

```
a = 15 ;
b =52 ;
c = 2*a – b ; d = b/a ;
```

2- Structures conditionnelles (alternatives)

i. Structure avec une alternative

Syntaxe : Formulation générale

```
if (<condition>) {
    <action>;
    ...
}
```

Exemple :

```
if (i == 5) {
    Somme = somme *5 ;
}
```

ii. Structure avec deux alternatives

Syntaxe : Formulation générale

```
if (<condition>) {
    <action>;
    ...
}
else {
    <action>;
    ...
}
```

Exemple :

```
if (i == 5) {
    Somme = somme *5;
}
else {
    Somme = somme /5 ;
}
```

iii. Structure avec alternatives reliées

Syntaxe : Formulation générale

```
if (<cond1> <opérateur> <cond2> ) {
    <action> ;
    <action> ;
    ...
} else {
    <action> ;
    ...
}
```

Exemple:

```
if (( i >= 10 ) && ( i <= 50 ) ) {
    somme = somme /5;
}
else {
    somme = somme +10;
}
```

iv. Structure avec conditions imbriquées

Syntaxe : Formulation générale

```
if (<condition1>) {
    <action>
} else {
    if (<condition2>) {
        <action>
    ...
}
```

```
} else {
    <action>
    ...
}
}
```

Exemple :

<pre> if (reponse ==1) { cadeau = "cigarettes"; } else { if (reponse==3) { cadeau = "fleurs" ; } </pre>	<pre> } else { cadeau = "chaussettes" ; } } </pre>
---	--

3- Structures répétitives (ou boucles ou itératives)

Il est très utile de disposer d'une instruction permettant d'effectuer de manière répétitive une série d'opérations. JavaScript propose plusieurs types de boucles itératives.

i. La boucle while

L'instruction **while** permet d'exécuter une instruction (ou un groupe d'instructions) un certain nombre de fois.

Syntaxe : Formulation générale

```

while ( <condition> ) {
    <action>;
    ...
}

```

Exemple :

```

a = 0 ; somme = 0 ;
while (a <25) {
    a = a+1 ;
    somme = somme + a ; }

```

Signification : Exécuter les <actions> aussi longtemps que la <condition> est vraie. Si la condition est fausse au début, aucune instruction n'est exécutée.

ii. La boucle do while

L'instruction **do ... while** permet d'exécuter une instruction (ou un groupe d'instructions) un certain nombre de fois.

Syntaxe : Formulation générale

```

<action> ;
...
( <condition> ) ;

```

Exemple :

```

a = 0 ; somme = 0 ;
do {
    a = a+1 ;
    somme = somme + a ;
}
while(a<25) ;

```

Signification Exécute les <actions> aussi longtemps que la <condition> est vraie. Si la condition est fausse au début, les instructions seront quand même exécutées une seule fois.

iii. La boucle for

Syntaxe : Formulation générale

```

for ( <expression de départ> ; <condition de continuation> ; <incrément> ) {
    <action>
    ...
}

```

Exemple :

```

somme = 0 ;
for (a=0; a < 25; a = a+1 ){
    somme = somme + a;
}

```


Signification : La boucle for se déroule de la manière suivante :

Tant que la condition de continuation est vraie c'est-à-dire ($a < 25$):

- en partant de l'expression de départ ($a = 0$) on exécute le contenu des accolades;
- la variable (a) est incrémentée ($a = a + 1$) et on exécute le contenu des accolades autant de fois que nécessaire;
- la boucle ne prend fin que lorsque la condition de continuation devient fausse c'est-à-dire lorsque a devient supérieur à 25.

Introduction

On appelle fonction un sous-programme qui permet d'effectuer un ensemble d'instruction par simple appel de la fonction dans le corps du programme principal. Cette notion de sous-programme est généralement appelée fonction dans les langages autres que le JavaScript (toutefois leur syntaxe est généralement différente...).

O.P.O : A la fin de cette leçon l'apprenant devra être capable de :

- ✓ Définir et utiliser une fonction dans un exercice
- ✓ Définir et utiliser une procédure dans un exercice
- ✓ Donner la différence entre une fonction et une procédure

I. Présentation générale

Les fonctions et les procédures permettent d'exécuter dans plusieurs parties du programme une série d'instructions, cela permet une simplicité du code et donc une taille de programme minimale. Dans JavaScript, les fonctions et les procédures sont définies par le mot clé **function**.

La différence entre une fonction et une procédure est que la fonction retourne une valeur (numérique, booléen ...) à la fin de son exécution et la procédure ne retourne aucune valeur. Ce retour de valeur se fait par le mot clé **return** qui est la dernière instruction de la fonction.

NB : Avant d'être utilisée, une fonction ou procédure doit être définie car pour l'appeler dans le corps du programme il faut que le navigateur la connaisse, c'est-à-dire son nom, ces arguments (paramètres) et les instructions qu'elle contient.

II. Syntaxe

La définition d'une fonction ou procédure s'appelle déclaration et se fait suivant la syntaxe suivante :

Syntaxe :

```
function nom_fonction(parametre 1, parametre 2, ...) {  
    <instructions >  
    <instructions >  
    .....  
    [ return resultat; ] /* présent si on une fonction */  
}
```


Exemple : fonction calculant le carrée d'un nombre qu'on lui passe en paramètre.

```
function carre(nombre){  
    res = nombre*nombre ;  
    return res;  
}
```

Pour utiliser une procédure , il suffit de faire appel à elle en écrivant son nom (tout en respectant la casse) suivie d'une parenthèse ouvrante (éventuellement ces arguments) puis d'une parenthèse fermante.

Syntaxe : Nom_Procedure () ;

Pour utiliser une fonction, il suffit de faire appel à elle en écrivant son nom (tout en respectant la case) suivie d'une parenthèse ouvrante (éventuellement ces arguments) puis d'une parenthèse fermante. Et la faire précéder d'une variable qui récupérera la valeur retournée.

Syntaxe : Nom_Variable = Nom_Fonction() ;

Exemple : a = carre(5) ;

III. Exemple de programme JavaScript

Pour illustrer tout ce qui a été fait jusqu'à ce niveau nous allons écrire un programme JavaScript qui permet de résoudre une équation du second degré. Nous allons écrire deux versions : l'une utilisant les procédures et fonctions et l'autre pas.

i. Programme sans fonctions ou procédures

```
<html>
<head> <title> résolution d'une équation du second degré </title>
</head>
<body>
  <script language="JavaScript" >
    alert('ce programme nous permettra de résoudre
une équation du second degré de la forme  $ax^2 + bx + c = 0$  ');

    a=prompt ("entrez la valeur de la variable a ");
    b=prompt ("entrez la valeur de la variable b ");
    c=prompt ("entrez la valeur de la variable c ");

    alert("l'équation à résoudre est : (" + a + ")x2 + (" + b +
")x + (" + c + ") = 0 ");

    if((a==0) && (b==0)){
      alert('c'est pas une équation du second degré');
    }
    else {
      if((a==0) && (b!=0)){ alert('s = { ' + -c/b + ' }' ); }
      else {
        if((a!=0) && (b==0)) {
          if ((-c/a)<0) {
            alert('impossible');
          }
        }
      }
    }
  </script>
</body>
</html>
```

```
    else {
      x1=Math.sqrt(-c/a);
      alert('S = { ' + x1 + ' }' );
    }
  }
  else {
    delta = b*b - 4*a*c;
    if(delta<0) {
      alert('ensemble vide'); }
    else {
      if(delta==0) { alert('S = { ' + -b/(2*a) + ' }' ); }
      else {
        x1 = (-b - Math.sqrt(delta))/(2*a);
        x2 = (-b + Math.sqrt(delta))/(2*a);
        alert('S = { ( ' + x1 + ' ; ' + x2 + ' ) }' );
      }
    }
  }
}
alert(" Fin d'exécution. Merci pour avoir testé ce programme !!!
Appuyer sur OK puis sur F5 pour recommencer ");
</script>
</body>
</html>
```

ii. Programme avec fonctions ou procédures

```
<html>
<head>
<title> Résolution d'une Equation du second degré </title>

  <script language="JavaScript" >
    function debut_programme(){
      alert('Ce programme nous permettra de résoudre
une équation du second degré de la forme  $ax^2 + bx + c = 0$  '); }
    function saisie_equation( ){
      a=prompt ("Entrez la valeur de la variable a ");
      b=prompt ("Entrez la valeur de la variable b ");
      c=prompt ("Entrez la valeur de la variable c "); }
    function affiche_equation(a, b, c){
      alert("L'équation à résoudre est : (" + a + ")x2 + (" + b +
")x + (" + c + ") = 0 "); }
    function determinant(a, b, c){
      delta = b*b - 4*a*c;
      return delta; }
    function determinant_positif(a, b, delta){
      x1 = (-b - Math.sqrt(delta))/(2*a);
      x2 = (-b + Math.sqrt(delta))/(2*a);
      alert('S = { ( ' + x1 + ' ; ' + x2 + ' ) }' ); }
    function fin_programme( ){
      alert(" Fin d'exécution. Merci pour avoir utilisé ce
programme !!! Appuyer sur OK puis sur F5 pour recommencer ");
    }
    function affiche_msg1( ){
      alert('C'est pas une équation du second degré'); }
    function affiche_msg2( ){
      alert('S = { ' + -c/b + ' }' ); }
  </script>
</body>
</html>
```

```
function affiche_msg3( ){
  alert('Impossible'); }
function affiche_msg4( ){
  alert('S = { ' + Math.sqrt(-c/a) + ' }' ); }
function affiche_msg5( ){
  alert('ensemble vide'); }
function affiche_msg6( ){
  alert('S = { ' + -b/(2*a) + ' }' ); }
</script>

</head>
<body>
  <script language="JavaScript">
    debut_programme( );
    saisie_equation( );
    affiche_equation(a, b, c);
    if((a==0) && (b==0)){ affiche_msg1( ); }
    else { if((a==0) && (b!=0)){ affiche_msg2( ); }
      else { if((a!=0) && (b==0)){
        if ((-c/a)<0){ affiche_msg3( ); }
        else { affiche_msg4( ); } } }
      else { delta = determinant(a, b, c);
        if(delta<0) { affiche_msg5(); }
        else { if(delta==0) { affiche_msg6(); }
          else { determinant_positif(a,b,delta); }
        } } } }
    fin_programme();
  </script>
</body>
</html>
```

iii. Résultat d'exécution



Figure 1 : Lancement de la page HTML

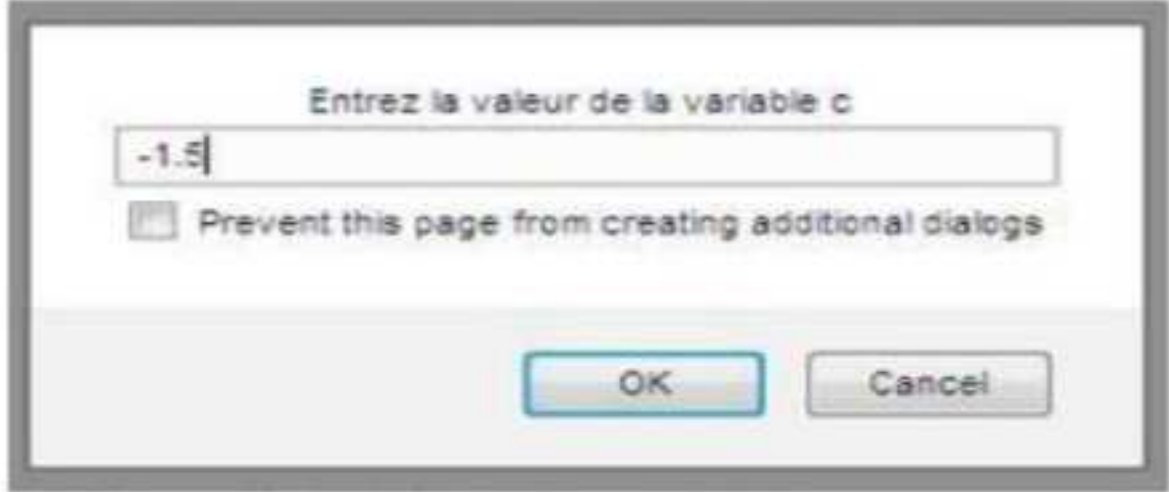
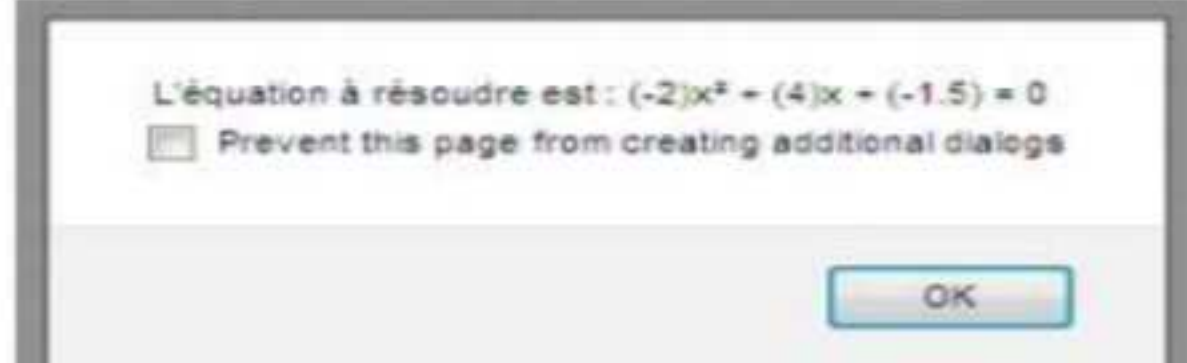
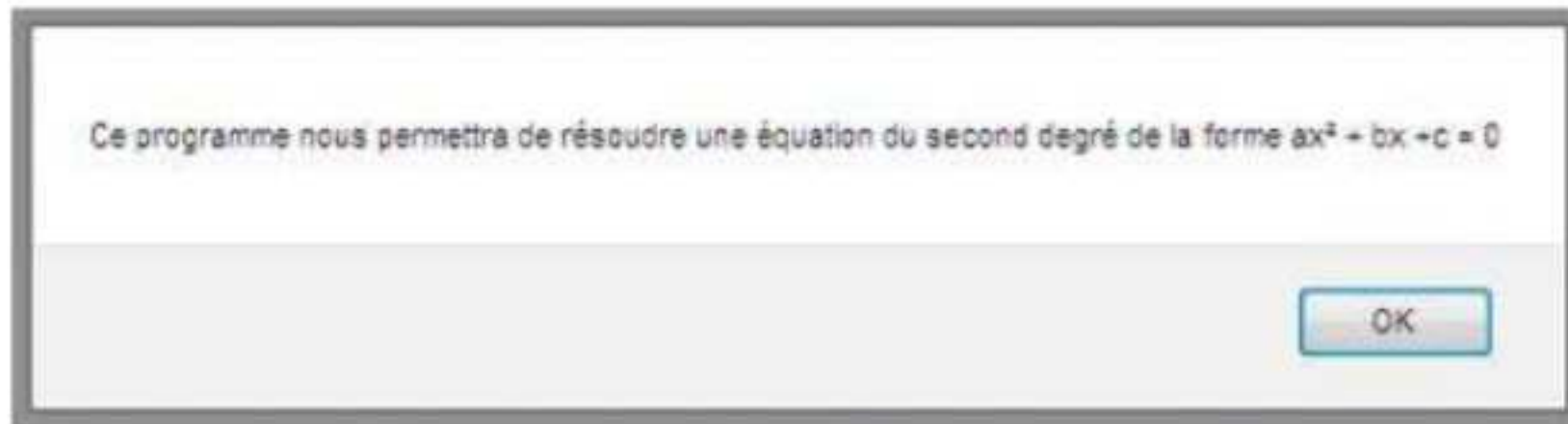


Figure 4 : Saisie de la variable « c »



iii. Résultat d'exécution



Ce programme nous permettra de résoudre une équation du second degré de la forme $ax^2 + bx + c = 0$

OK

Figure 1 : Lancement de la page HTML



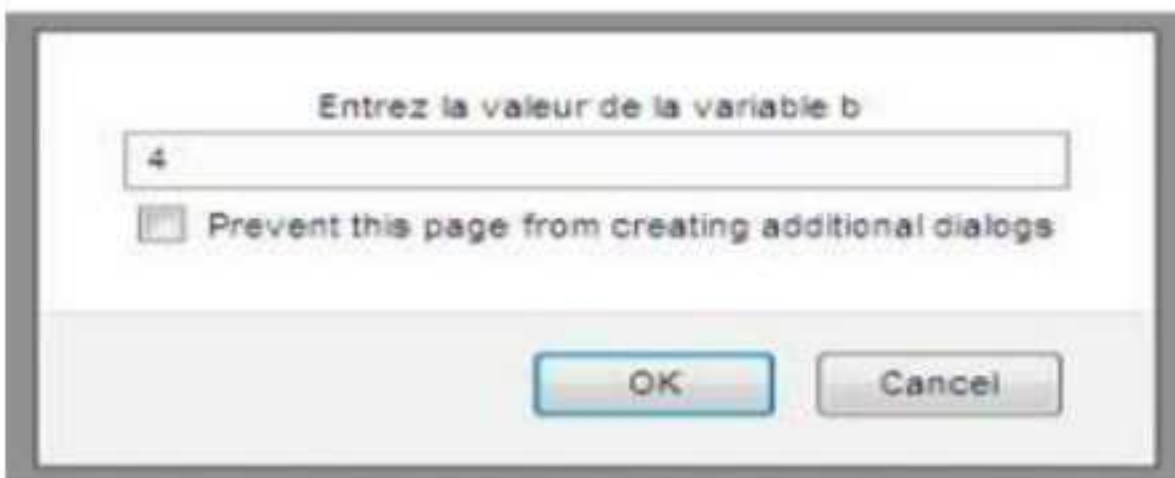
Entrez la valeur de la variable a

-2

☐ Prevent this page from creating additional dialogs

OK Cancel

Figure 2 : Saisie de la variable « a »



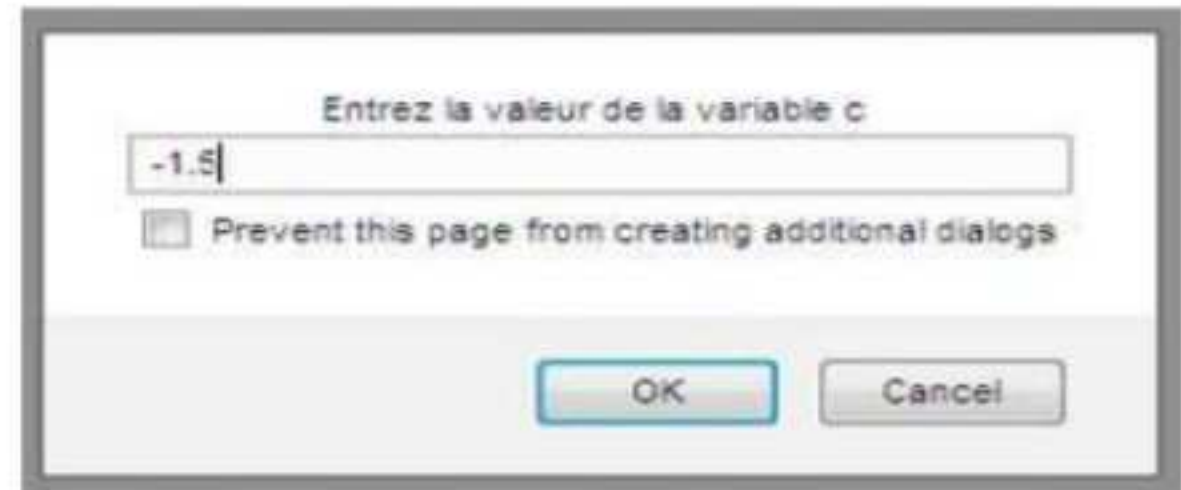
Entrez la valeur de la variable b

4

☐ Prevent this page from creating additional dialogs

OK Cancel

Figure 3 : Saisie de la variable « b »



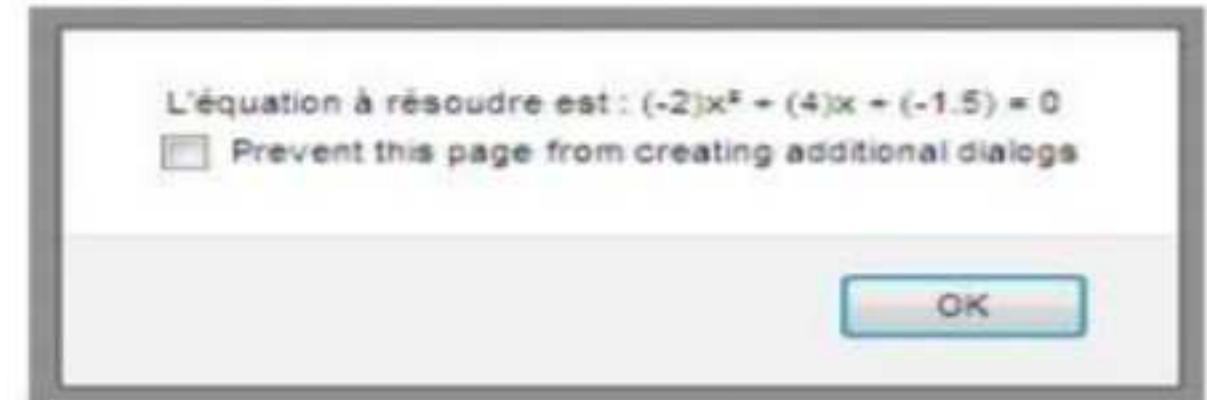
Entrez la valeur de la variable c

-1.5

☐ Prevent this page from creating additional dialogs

OK Cancel

Figure 4 : Saisie de la variable « c »



L'équation à résoudre est : $(-2)x^2 + (4)x + (-1.5) = 0$

☐ Prevent this page from creating additional dialogs

OK

Figure 5 : Affichage de l'équation saisie



S = { (1.5 ; 0.5) }

☐ Prevent this page from creating additional dialogs

OK

Figure 6 : Affichage du résultat d'exécution



Fin d'exécution. Merci pour avoir utilisé ce programme. Appuyer sur OK puis sur F5 pour recommencer

☐ Prevent this page from creating additional dialogs

OK

Figure 7 : Affichage Message de fin d'exécution